

Multithreaded Web Crawler Pipeline (Crawler + Indexer + Query)

Project Overview

You will build a scalable, concurrent web crawler that downloads pages in parallel and feeds them through a bounded pipeline to construct a simple persistent on-disk inverted index. You will also implement a query tool that searches the index.

Pipeline:

URLs → Fetch HTML → Parse links + extract text → Save document → Send metadata via IPC → Build inverted index → Query

High-Level Architecture

Your system must be split into at least two processes (crawler + indexer). A third program (query) searches the built index.

1 Crawler (process)

- Maintains a bounded, thread-safe URL frontier (work queue) with backpressure.
- Runs a fixed-size worker thread pool (no thread-per-URL).
- Fetches pages using libcurl and persists each fetched page (or extracted text) to disk.
- Parses HTML to extract outgoing links; normalizes/filters to HTTP(S); enqueues new URLs subject to constraints.
- Sends document metadata to the indexer via IPC: (docid, url, filepath, depth).

2 Indexer (process)

- Receives document metadata messages from the crawler via IPC.
- Reads saved pages (or saved extracted text), tokenizes into words, and updates a persistent inverted index on disk.
- Maintains a document map: docid → url + filepath.
- Flushes index structures to disk so the index remains consistent after program exit.

3 Query (program)

- Loads dictionary/index metadata from disk.
- Retrieves postings lists for one or more query terms.
- Outputs matching docids and URLs (AND across terms is a sufficient baseline).

Core Requirements

Requirement	Implementation Goals
Crawler: Thread pool	Fixed-size pthread worker pool; threads are reused and terminate gracefully.
Crawler: Bounded URL queue	Thread-safe bounded queue using mutex + condition variables; producers block when full; consumers block when empty; supports shutdown wakeups.
Crawler: Visited set	Thread-safe visited set to avoid re-crawling the same URL; check+insert atomically.
Crawler: Fetching + robustness	Use libcurl; handle timeouts/HTTP errors and file I/O failures without crashing; log failures.
Crawler: Stop conditions	Race-free stop conditions for --max-depth and --max-pages (and/or frontier exhaustion).
Crawler: Persistence	Persist downloaded pages under an output directory using a unique docid-based naming scheme.
IPC	Crawler and indexer communicate via UNIX domain socket OR pipe/FIFO with a clearly defined message format.
Indexer: Persistent inverted index	Build an on-disk inverted index (word → postings list of docids/URLs) plus a document map.
Query	Command-line tool to search the on-disk index; supports multi-term AND queries at minimum.
Logging + summary	Log progress and print final summary: pages fetched, skipped, failed, max queue depth, runtime.

Command-Line Interface

Use a CLI similar to the following (you may add more flags as needed):

```
crawler --seed <url> --max-depth <D> --max-pages <N> -t <threads> --out <dir> --ipc <path>
indexer --ipc <path> --out <dir>
query --index <dir> <term1> [term2 ...]
```

Example run sequence:

```
./indexer --ipc /tmp/crawl.sock --out data/index
./crawler --seed https://example.com --max-depth 3 --max-pages 1000 -t 12 --out data --ipc
/tmp/crawl.sock
./query --index data/index operating systems threads
```

Example usage

Help prompt:

```
./crawler -h
```

Output:

```
USAGE: crawler --seed <url> --max-depth <D> --max-pages <N> -t <threads> --out <dir> --ipc  
<path>
```

End-to-end run (start indexer, crawl, then query):

```
./indexer --ipc /tmp/crawl.sock --out data/index  
./crawler --seed https://en.wikipedia.org/wiki/Linux --max-depth 3 --max-pages 500 -t 12 --out  
data --ipc /tmp/crawl.sock  
./query --index data/index operating systems threads
```

Sample query output:

```
Found 3 matching documents (AND across terms):  
42 https://en.wikipedia.org/wiki/Operating_systems  
87 https://en.wikipedia.org/wiki/Thread_(computing)  
105 https://en.wikipedia.org/wiki/POSIX_Threads
```

No results within the current index:

```
./query --index data/index thistermdoesnotexist
```

Output:

```
No documents matched all query terms.
```

On-Disk Data Layout (suggested)

- pages/ saved HTML (or extracted text) files named by docid
- index/ directory for index files
- index/docs.tsv document map: docid<TAB>url<TAB>filepath<TAB>depth
- index/dict.tsv dictionary: term<TAB>offset<TAB>df (or any equivalent metadata you choose)
- index/postings.bin or postings.tsv postings lists stored in an efficient or simple format

You may choose a different layout, but it must be documented and persist across program runs.

Deliverables

- Implement using C, C++
- Makefile with targets: all, clean, run.
- Source code for crawler, indexer, and query (multiple .c/.h files allowed).
- README.md explaining: build/run steps, queue/visited design, IPC protocol, index format, and stretch features (if any).